WEEK 9

Accelerators

HPC & Cluster Computing

Course: Operating Systems 4 (HPC)
Module: EIOSY4A
Topic: GPU Accelerators and Heterogeneous Computing
Reading Time: ~15 Minutes

Learning Outcomes
By the end of this week, you should be able to:
1. **Evaluate** Heterogeneous Computing models and justify the use of GPUs for massively parallel workloads.
2. **Analyze** deep GPU architectural features including Streaming Multiprocessors, the SIMT model, and warp scheduling.
3. **Design** parallel kernels using the CUDA programming model, mapping threads, blocks, and grids.
4. **Critique** the specific HPC use cases for FPGAs and specialized ASICs versus general-purpose GPUs.
5. **Analyze** the PCIe data transfer bottleneck and formulate strategies to minimize host-to-device latency.

Week Goal
Welcome to Week 9! We leave the CPU behind and explore the hardware that powers the modern AI and simulation revolution. GPUs are not just for gaming - they are the dominant force in the Top500 list and in AI research.

WEEK 8 WEEK 10

Assessment Alignment: The weekly quiz and practical assignments for this module specifically evaluate the Learning Outcomes listed below, verifying your ability to operate at an NQF Level 7 standard of analysis and synthesis.

Overview

For decades, Moore's Law drove performance improvements in CPUs. However, as we hit physical limits (power wall, heat), simply boosting clock speeds became impossible. This led to the era of **Heterogeneous Computing**: using specialized hardware alongside the CPU to accelerate specific tasks. In this chapter (referencing Sterling Chapter 15), we explore **Accelerators**—specifically GPUs—which power the majority of the world's top supercomputers today.

This content strictly follows **Sterling Chapters 15 (Architecture) and 16 (Programming)**

9.1 Taxonomy of Accelerators

9.1.1 What is an Accelerator?

An **Accelerator** is a specialized hardware component designed to offload specific, compute-intensive tasks from the general-purpose main CPU (often called the **Host**). While a CPU is a "Jack of all trades"—optimized for running operating systems, handling interrupts, and executing complex serial logic—it is not efficient at processing massive blocks of data in parallel.

Accelerators (the **Device**) differ by dedicating nearly all their transistors to arithmetic processing units (ALUs) rather than cache or control logic. By offloading parallel-heavy workloads (like matrix multiplication or simulations) to the accelerator, the system achieves significantly higher throughput and energy efficiency than using the

9.1.2 Types of Accelerators

Type	Name	Flexibility	Efficiency	Example
GPU	Graphics Processing Unit	High	High (for parallel tasks)	NVIDIA H100, AMD MI300
FPGA	Field-Programmable Gate Array	Medium (Reconfigurable)	Very High	Intel Stratix, Xilinx Alveo
ASIC	Application-Specific Integrated Circuit	Low (Fixed hardware)	Maximum	Google TPU, Bitcoin Miners
Manycore	Many Integrated Core	High (Like a CPU)	Moderate	Intel Xeon Phi (Legacy)

In HPC, **GPUs** are the dominant accelerator because they offer a sweet spot of programmability and massive raw performance.

9.2 GPGPU Architecture

Originally designed for rendering video games (pixels), GPUs turned out to be excellent for matrix math, which is the foundation of scientific simulation (and AI). This usage is called **GPGPU** (General-Purpose computing on Graphics Processing Units).

9.2.1 CPU vs. GPU Design Philosophy

The design philosophy of a **CPU** is **Latency Oriented**. A CPU is architected to finish a single task (thread) as quickly as possible. To achieve this, it dedicates the vast majority of its silicon to ma[illegible] and complex control logic (branch prediction, out-of-order e[illegible]). This is ideal for operating systems, serial logic, and tasks with comple[illegible].

GPU is **Throughput Oriented**. It is designed to maximize the total volume of work completed, executing thousands of threads simultaneously, even if individual thread speed is slow. It sacrifices deep caches and branch prediction in favor of packing thousands of tiny arithmetic cores onto the die, making it perfect for massive data parallelism where the same operation is performed on millions of pixels or floating-point numbers.

9.2.2 Key Components

The computational engine of a GPU is the **Streaming Multiprocessor (SM)**, which functions roughly like a CPU core but with a crucial difference: each SM contains hundreds of simpler "CUDA Cores" (ALUs). These cores are fed by **High Bandwidth Memory (HBM)**, a specialized stacked memory technology (e.g., HBM3) located directly on the GPU application substrate. HBM provides immense data throughput (3+ TB/s) compared to standard system RAM (100 GB/s), preventing the massive array of cores from starving for data.

To manage this parallelism, GPUs use an execution model called **SIMT (Single Instruction, Multiple Thread)**. Instead of decoding an instruction for every thread, the GPU issues a single instruction (e.g., "Add Value A + Value B") to a group of 32 threads known as a **Warp**. All threads in the warp execute this same instruction simultaneously but on different data points, amortizing the cost of instruction decoding.

9.2.3 Performance Metrics

To evaluate if a GPU is being used efficiently, we track two primary mathematical metrics:

- **Arithmetic Intensity (I)**: The ratio of floating-point operations ($FLOPs$) to bytes of data moved from memory ($Bytes$):

$$I = \frac{FLOPs}{Bytes}$$

If I is low, the GPU is waiting for data (Memory Bound). If I is high, the GPU is busy calculating (Compute Bound).

- **Occupancy (Ω):** The ratio of active warps to the maximum possible warps on an SM processor.

$$\Omega = \frac{\text{Active Warps}}{\text{Maximum Warps}}$$

> Higher occupancy allows the GPU to "hide" memory latency by switching to another warp while one is waiting for data.

9.3 Programming Accelerators

Standard sequential C/C++ code cannot execute directly on a GPU. Instead, the application must be architected to explicitly distinguish between the **Host** (CPU) control logic and the **Device** (GPU) computational kernels.

9.3.1 Coding Models

CUDA (Compute Unified Device Architecture) is the most widely used proprietary model, developed by NVIDIA. It is the distinct gold standard for HPC and AI, allowing developers to write C++ with extensions that grant granular low-level control over memory handling and thread management.

For open solutions, **OpenCL** (and the newer **HIP** by AMD) offers a vendor-neutral standard. Tools like HIPIFY can even translate CUDA source code into portable HIP code, allowing deployment on non-NVIDIA hardware.

Finally, for ease of use, directive-based models like **OpenACC** and **OpenMP Offloading** allow developers to simply annotate existing C/C++ loops with `#pragma` directives. The compiler then handles the complex data movement and kernel generation automatically, though this convenience sometimes comes at the cost of peak performance compared to hand-tuned CUDA.

9.3.2 The Host-Devi

C++ Example (CUDA Pseudo-code)

```
// 1. Allocate Memory on GPU (VRAM)
cudaMalloc(&d_A, size);

// 2. Transfer Data (Host RAM -> GPU VRAM)
cudaMemcpy(d_A, h_A, size, cudaMemcpyHostToDevice);

// 3. Launch Kernel (Compute)
// <<<Grid, Block>>> syntax launches thousands of threads
matrixMultiply<<<numBlocks, threadsPerBlock>>>(d_A, d_B,

// 4. Retrieve Results (GPU VRAM -> Host RAM)
cudaMemcpy(h_C, d_C, size, cudaMemcpyDeviceToHost);
```

Explanation: The CPU acts as a traffic controller. It moves data across the PCIe bus (Section A & D) and triggers the massive parallel work (Section C). Note that "Compute" is fast, but "Transfer" is slow (limited by PCIe bandwidth).

9.3.3 CUDA Thread Hierarchy

Valid GPU kernels must map to the hardware's hierarchy. To achieve this, CUDA divides the work into a three-level structure: **Grids**, **Blocks**, and **Threads**.

The **Grid** represents the entire problem space (e.g., all 4000×4000 pixels in an image). This Grid is divided into smaller chunks called **Blocks**. A Block is a logical group of threads that run on a single Streaming Multiprocessor (SM). Finally, each **Thread** is the smallest unit of execution, processing a single data point. The programmer must define `dim3 blocks` and `dim3 threads` to map their problem size to this grid, ensuring that every data element is covered by a thread.

High-performance GPU programming requires manual management of the memory hierarchy. Unlike a CPU which manages cache automatically, a GPU programmer usually manages it explicitly to achieve peak performance.

Global Memory is the largest memory space (the 16GB-80GB of VRAM) accessible by all threads, but it has high latency and is relatively slow. In contrast, **Shared Memory** is a tiny, incredibly fast slice of memory located on the SM itself, shared only by threads within the same Block. A common optimization pattern is for threads to cooperatively load a chunk of data from Global Memory into Shared Memory, perform heavy calculations on the fast Shared Memory, and then write the final result back to Global.

9.4 Modern Trends

9.4.1 AI and Tensor Cores

Modern GPUs include specialized headers called **Tensor Cores**. These are hardwired ASICs inside the GPU designed to do just one thing: Matrix Multiplication ($A \times B + C$) for 4×4 or 16×16 matrices. This is the math behind Deep Learning, making GPUs the engine of the AI revolution.

9.4.2 Multi-GPU Interconnects

The traditional PCIe bus is often a bottleneck for multi-GPU training. To overcome this, NVIDIA introduced **NVLink**, a high-speed direct interconnect that allows GPUs to communicate with each other at blazing speeds without burdening the CPU or traversing the system bus. For massive clusters, **NVSwitch** extends this concept, connecting dozens or hundreds of GPUs into a single unified high-bandwidth fabric, effectively treating them as one giant accelerator.

5. Summary and Next Steps

This week, we explored the world of **Accelerators**, specialized hardware designed to bypass the physical limitations of modern CPUs. We learned that while CPUs are latency-optimized for complex logic, GPUs are throughput-optimized for massive parallelism. By understanding the **Host-Device** model, the **SIMT** execution pattern, and the importance of **High Bandwidth Memory (HBM)**, you are now equipped to navigate the heterogeneous architecture of modern Exascale systems.

Preparing for Week 11

Next week (skipping the deprecated Week 10), we will bring everything together in **Real World Applications**. We will look at the actual scientific codes—from Weather Forecasting to Genomics—that run on the clusters we have designed, provisioned, and programmed. We will:

- Examine case studies like **WRF** (Weather) and **GROMACS** (Molecular Dynamics).
- Understand the **Exascale Challenges** of power and concurrency.
- Begin planning your **Capstone Project**.

Checklist:

- Why is a GPU characterized as "Throughput Oriented" while a CPU is "Latency Oriented"?
- What is a **Warp** in CUDA terminology, and what happens if threads in a warp diverge?
- Why is **PCIe** often the bottleneck in accelerator performance, and how does **NVLink** solve this?
- Can you explain the difference between an **FPGA** (Reconfigurable) and an **ASIC** (Fixed)?

Resources

- Sterling, T., Anderson, M. and Brodowicz, M. (2018) *High Performance Modern Systems and Practices*. Morgan Kaufmann. Chapter 19.
- NVIDIA CUDA Programming Guide

Test Your Knowledge

Ready to check your understanding of this week's material? Take the interactive quiz now!

Start Quiz